Autonomous Edge Robotics: A Comparative Analysis of Behavioral Cloning and Deep Q-Networks in Constrained 2D Environments

Prashanna Tiwari

Abstract

The development of lightweight, reactive navigation systems for mobile edge robotics is a critical challenge in modern computer science. Traditional pathfinding relies on heavy, map-based computations which are often unsuited for hardware-constrained edge devices. This paper investigates the efficacy of deploying artificial neural networks to map raw sensor data directly to motor commands, bypassing traditional algorithmic pathfinding. We conduct a rigorous comparative study between two distinct machine learning paradigms: Behavioral Cloning (a supervised imitation learning approach) and Deep Q-Networks (a model-free reinforcement learning approach). Notably, to test the specific strengths of each paradigm, the supervised model navigated a continuous space with multiple randomized rectangular obstacles, while the reinforcement learning agent was tasked with finding a narrow pass-through corridor in a discrete-grid framework. Our findings indicate that while Behavioral Cloning is highly efficient at replicating hard-coded heuristic safety rules, the reinforcement learning agent—guided by potential-based reward shaping and Double-DQN target updates—develops a superior capacity for complex spatial goal-seeking. Furthermore, this paper provides a deep mathematical analysis of observed training failures, specifically addressing the mechanics of reward plateaus and vanishing temporal-difference errors, and proposes concrete architectural improvements for future iterations.

Contents

1 Introduction

1.1 Motivation and Background

In the rapidly evolving field of robotics, autonomous navigation remains a cornerstone problem. For a robot to move safely from a starting point to a destination, it must perceive its environment, identify obstacles, and execute precise motor commands. Historically, this has been achieved using Simultaneous Localization and Mapping (SLAM) alongside algorithms like A* or Dijkstra's [1]. However, these methods require significant computational overhead, memory, and an absolute global coordinate system.

For "edge robotics"—small, low-cost robots with limited processing power—these traditional methods are often infeasible. This has led to a surge in interest regarding reactive navigation policies powered by Machine Learning (ML). By training Artificial Neural Networks (ANNs) to interpret local sensor data (such as LiDAR or ultrasonic distances) and output direct motor velocities, we can achieve high-speed inference with minimal computational cost.

1.2 Project Scope and Objectives

This project seeks to build an end-to-end simulation, training, and evaluation pipeline for reactive robot navigation. We aim to answer a fundamental question:

> *Is it more effective to teach a robot by providing it with explicit examples of good behavior (Supervised Learning), or by allowing it to learn through trial, error, and delayed rewards (Reinforcement Learning)?*

To answer this, we developed a Pygame-based 2D physics simulation and implemented two distinct training architectures:

1. **Behavioral Cloning (BC):** A supervised imitation learning framework where a multi-layer perceptron mimics a deterministic, rule-based expert controller, building upon foundational concepts introduced by Pomerleau's ALVINN [2]. We designed a programmatic, rule-based "expert" algorithm, let it navigate the simulation, recorded its sensor inputs and motor outputs, and trained a network to mimic it.
2. **Deep Q-Networks (DQN):** A model-free reinforcement learning approach where an agent interacts natively with an unknown environment, optimizing a parameter-

ized value function via a reward signal, utilizing architectures modernized by Mnih et al. [3]. The agent begins with no prior knowledge of the world; by defining a mathematical reward function based on goal proximity and collision penalties, it learns an optimal policy via reinforcement.

1.3 Document Structure

The remainder of this paper is structured as follows. Section 2 details the mathematical formulation of the simulation environments and sensor kinematics. Section 3 explores the theoretical and practical application of Behavioral Cloning. Section 4 provides a deep dive into the Markov Decision Process and Deep Q-Network implementation. Section 5 presents our experimental results. Section 6 provides a rigorous mathematical analysis of the problems faced during training, specifically the reward plateau phenomenon. Finally, Section 7 outlines proposed future improvements.

2 Simulation Environments and Kinematics

To properly evaluate both learning paradigms, we engineered custom 2D simulation environments rather than relying on heavy pre-built physics engines. This allowed for total control over state observation and reward generation. The supervised learning model required a dense obstacle field to generate diverse avoidance data, while the reinforcement learning model required a constrained puzzle to test goal-seeking capabilities.

2.1 Environment A: Continuous Multi-Obstacle Space (Behavioral Cloning)

For the Behavioral Cloning dataset generation, we built a Pygame environment operating in a continuous 800×600 coordinate space. The environment features outer boundary walls and multiple inner geometric obstacles, including standard rectangular blocks and a hollow "box" structure.

The robot's pose is characterized by $(x, y) \in \mathbb{R}^2$ and heading θ. It perceives the world using three raycasts at angles θ, $\theta + 0.5$, and $\theta - 0.5$ radians. The maximum raycasting distance is set to 200 units. To ensure neural network stability, raw distances D_i are normalized as follows:

$$\text{Sensor}_i = 1 - \frac{D_i}{200} \tag{1}$$

Uniform noise $\mathcal{U}(-0.01, 0.01)$ is injected into the sensor readings to simulate real-world sensor imperfections.

2.2 Environment B: Discrete Grid with Narrow Corridor (DQN)

For the reinforcement learning task, we constructed a 10×10 discrete matrix grid world. The environment is defined using binary occupancy encoding:

$$W_{i,j} = \begin{cases} 1 & \text{if cell } (i, j) \text{ contains an obstacle} \\ 0 & \text{if cell } (i, j) \text{ is free space} \end{cases} \tag{2}$$

To introduce a complex navigation challenge requiring spatial awareness rather than simple straight-line movement, a horizontal barrier is permanently affixed at row $y = 5$.

A narrow two-cell corridor is hollowed out at the centre of the barrier to allow traversal:

$$W_{5,\,x} = 1 \quad \forall\, x \notin \{4,\, 5\} \tag{3}$$

$$W_{5,\,4} = 0, \qquad W_{5,\,5} = 0 \tag{4}$$

2.3 Robot Pose and Initialization

The robot's pose is not locked to the discrete grid; it operates in continuous space. The state is characterized by the tuple (x, y, θ), where $x, y \in \mathbb{R}$ are Cartesian coordinates and $\theta \in [0, 2\pi)$ is the heading angle in radians. To ensure robust training across a diverse set of starting conditions, the episode reset function initializes the robot and the goal at random coordinates within free cells, centred within the chosen cell:

$$x_{\text{init}} = x_{\text{grid}} + 0.5, \qquad y_{\text{init}} = y_{\text{grid}} + 0.5 \tag{5}$$

$$\theta_{\text{init}} \sim \mathcal{U}(0,\, 2\pi) \tag{6}$$

2.4 Raycast Sensor Physics

The robot perceives the world using a simulated form of LiDAR or ultrasonic sensing, implemented via raycasting. The agent is equipped with three rays whose angular orientations relative to the current heading θ are:

$$\alpha_{\text{front}} = \theta, \qquad \alpha_{\text{left}} = \theta + 0.5 \text{ rad}, \qquad \alpha_{\text{right}} = \theta - 0.5 \text{ rad} \tag{7}$$

For each ray, the distance to the nearest obstacle is computed using an incremental marching algorithm with step size $\Delta = 0.5$. The ray tip's position at step k advances as:

$$x_{k+1} = x_k + \cos(\alpha_i) \cdot \Delta, \qquad y_{k+1} = y_k + \sin(\alpha_i) \cdot \Delta \tag{8}$$

Marching terminates when the ray intersects a blocked cell. The accumulated distance $D_i = \sum \Delta$ is then normalized before being fed into the network:

$$\text{Normalized Sensor}_i = \frac{D_i}{\text{Max Range}} \tag{9}$$

Both environments implement this identical raycasting model; however, their visual layouts differ significantly. Figure 1 shows Environment A (the continuous multi-obstacle space used for Behavioral Cloning data collection), while Figure 2 shows Environment B (the discrete grid world with the narrow central corridor used for DQN training).

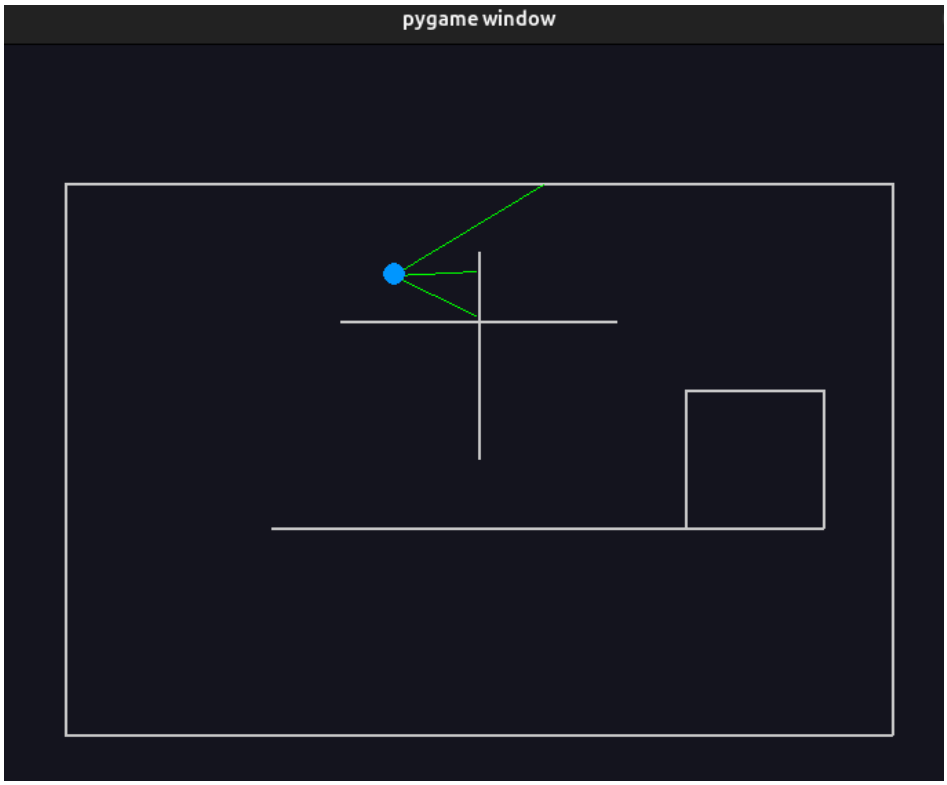

Figure 1. Environment A: Pygame simulation of the continuous multi-obstacle space. The robot emits three raycast lines toward the surrounding rectangular obstacles used for Behavioral Cloning data collection.

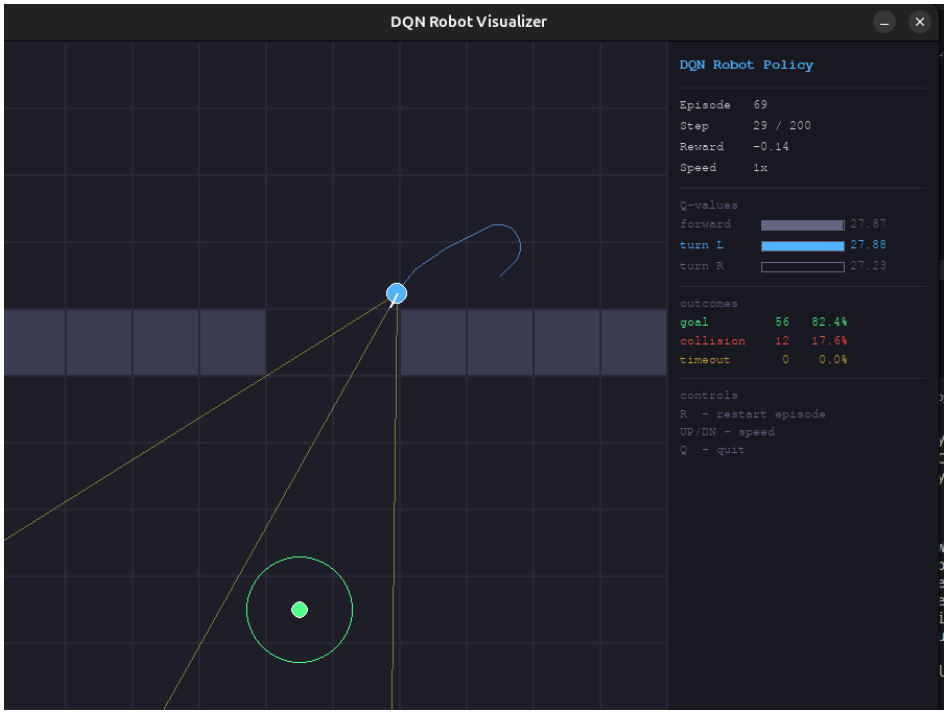

Figure 2. Environment B: Pygame simulation of the discrete 10×10 grid world. The horizontal wall with the narrow two-cell corridor at $x \in \{4, 5\}$ is clearly visible, along with the three raycast lines used by the DQN agent.

3 Imitation Learning: Behavioral Cloning

Behavioral Cloning (BC) frames the navigation problem as a supervised learning task. The objective is to learn a mapping function $f(x) \approx y$, where x is the state observation and y is the action taken by an algorithmic expert.

3.1 Expert Trajectory Generation

To generate training data, we implemented a hard-coded, rule-based differential-drive controller. The expert continuously drives forward with a base velocity of $v_L = 0.7$, $v_R = 0.7$, and modifies these velocities according to a strict priority hierarchy:

1. **Corner Trap Detection (Highest Priority):** If all three sensors detect an obstacle in close proximity ($S_{\text{front}} > 0.4 \ \wedge \ S_{\text{left}} > 0.4 \ \wedge \ S_{\text{right}} > 0.4$), the robot executes a hard pivot in place: $v_L = 0.8$, $v_R = -0.8$.
2. **Frontal Obstacle Avoidance:** If an obstacle is detected ahead ($S_{\text{front}} > 0.3$), the robot evaluates the side sensors and pivots away from the side with the highest proximity reading.
3. **Side Cushioning (Lowest Priority):** If the robot drifts too close to a side wall ($S_{\text{left}} > 0.4$ or $S_{\text{right}} > 0.4$), it gently nudges the opposite motor to re-centre itself.

Uniform noise $\mathcal{U}(-0.01, 0.01)$ was injected into the sensor readings during generation, and a safe-respawn mechanic was employed to avoid collecting static data during crashes. In total, 100,000 labelled state-action pairs were logged via batch logging.

3.2 Supervised Training Architecture

The generated dataset is processed using Pandas with an 80/20 train-validation split. The predictive model is built in PyTorch as a dense, feed-forward multi-layer perceptron (MLP). Three sensor readings serve as input and are passed through a funnelling structure of hidden layers ($32 \to 32 \to 16$ neurons) using Rectified Linear Unit (ReLU) activations to capture non-linear relationships:

$$f(x) = \max(0,\, x) \tag{10}$$

The final output layer yields 2 values representing the left and right motor velocities. A

Tanh activation bounds the outputs to $[-1, 1]$. Network weights are optimized using the Adam optimizer with mini-batches of size 64, minimizing the Mean Squared Error (MSE) loss:

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N} \sum_{i=1}^{N} \left(y_{\text{expert}}^{(i)} - \hat{y}_{\text{network}}^{(i)} \right)^2 \tag{11}$$

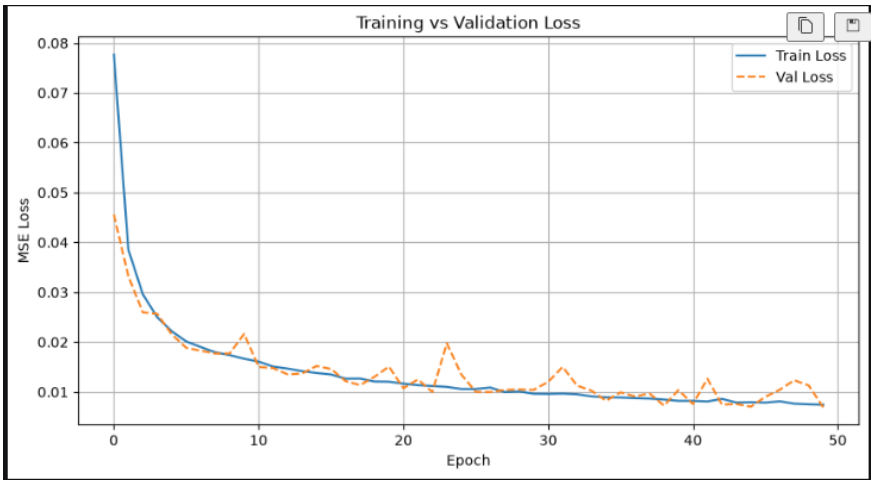

Figure 3. Behavioral Cloning training vs. validation loss over 50 epochs, demonstrating smooth convergence without overfitting.

4 Reinforcement Learning: Deep Q-Networks

While Behavioral Cloning is straightforward to train, its performance is strictly capped by the quality of the expert. Reinforcement Learning (RL) allows the agent to potentially surpass human-designed heuristics by maximizing an objective mathematical reward [4].

4.1 Markov Decision Process (MDP) Formulation

We model the navigation task as a Markov Decision Process, defined by the tuple $\langle S, A, P, R, \gamma \rangle$.

State Space (S). The observation vector fed to the neural network consists of 7 continuous, normalized variables:

$$S = \left[\frac{r_0}{10}, \ \frac{r_1}{10}, \ \frac{r_2}{10}, \ \frac{d_{\text{goal}}}{14.0}, \ \sin(\theta), \ \cos(\theta), \ \frac{\Delta\theta}{\pi} \right] \tag{12}$$

Here r_i are the raycast distances, d_{goal} is the Euclidean distance to the goal, and $\Delta\theta$ is the relative angle between the robot's heading and the goal direction.

Action Space (A). Unlike the continuous motor outputs of the BC model, the DQN agent selects from a discrete set of three actions:

- a_0: Translate forward (0.3 units).
- a_1: Rotate left ($+0.15$ rad) and translate forward (0.05 units).
- a_2: Rotate right (-0.15 rad) and translate forward (0.05 units).

4.2 Dense Reward Shaping

A naive reward function that only grants $+1$ for reaching the goal creates a "sparse reward" problem, where the agent wanders aimlessly for thousands of episodes without receiving a useful learning signal. To solve this, we implemented potential-based dense reward shaping [5]. At every timestep t, the change in Euclidean distance to the goal provides a continuous incentive gradient:

$$\text{shaping} = d_t - d_{t+1} \tag{13}$$

The full composite reward structure is:

$$R = \begin{cases} 20.0 + (T_{\max} - t) \cdot 0.05 & \text{if goal reached} \\ -10.0 & \text{if collision} \\ \text{shaping} + 0.02 & \text{if } a = a_0 \text{ (encourage forward motion)} \\ \text{shaping} - 0.05 & \text{if } a \in \{a_1, a_2\} \text{ (discourage spinning)} \end{cases} \tag{14}$$

4.3 The Double-DQN Architecture

The agent approximates the optimal action-value function $Q^*(s, a)$ using a neural network $Q(s, a;\, \phi)$. The Q-Network consists of fully connected linear layers ($7 \to 256 \to 256 \to 128 \to 3$) augmented with Layer Normalization and LeakyReLU (slope 0.01). Layer Normalization is crucial for stabilizing learning dynamics against the shifting distribution of continuous state inputs.

An Experience Replay buffer (capacity 100,000) is used to break temporal correlations in sequential data, with random mini-batches of size 64 sampled for each training step.

To mitigate the well-documented overestimation bias of standard Q-learning, we utilize a Double-DQN methodology [6]. Two networks are maintained: an Online Network (ϕ) and a Target Network (ϕ^-). The Online Network *selects* the best next action; the Target Network *evaluates* it:

$$Y_t = r_t + \gamma\, Q\Big(s_{t+1},\ \arg\max_{a'} Q(s_{t+1}, a';\, \phi);\ \phi^-\Big) \tag{15}$$

Loss is computed using the Smooth L1 (Huber) Loss, which is less sensitive to outliers than MSE:

$$L(\phi) = \begin{cases} 0.5\,(Y_t - Q)^2 & \text{if } |Y_t - Q| < 1 \\ |Y_t - Q| - 0.5 & \text{otherwise} \end{cases} \tag{16}$$

Target Network weights are updated smoothly at every step using Polyak averaging with $\tau = 0.005$:

$$\phi^- \ \leftarrow\ \tau\phi + (1 - \tau)\phi^- \tag{17}$$

4.4 Algorithmic Implementation

The complete training procedure, including experience replay and soft target updates, is presented in Algorithm 1.

Algorithm 1 Double Deep Q-Network Training with Soft Target Updates

Initialize replay memory D to capacity 100,000
Initialize online network Q with random weights θ
Initialize target network $\hat{Q}$ with weights $\theta^- \leftarrow \theta$
for episode $= 1$ **to** M **do**
    Reset environment; obtain initial state s_1
    for $t = 1$ **to** T **do**
        With probability ϵ select random action a_t; otherwise $a_t = \arg\max_a Q(s_t, a; \theta)$
        Execute a_t; observe reward r_t and next state s_{t+1}
        Store transition (s_t, a_t, r_t, s_{t+1}) in D
        if $|D| >$ MinReplaySize **then**
            Sample random minibatch $\{(s_j, a_j, r_j, s_{j+1})\}$ from D
            $a'_{\max} \leftarrow \arg\max_a Q(s_{j+1}, a; \theta)$
            $y_j \leftarrow r_j + \gamma \hat{Q}(s_{j+1}, a'_{\max}; \theta^-) \cdot (1 - \text{done})$
            Gradient descent on HuberLoss$(y_j, Q(s_j, a_j; \theta))$
            $\theta^- \leftarrow \tau\theta + (1 - \tau)\theta^-$
        end if
        $s_t \leftarrow s_{t+1}$
    end for
    $\epsilon \leftarrow \epsilon \times 0.9999$
end for

5 Experimental Results

The Deep Q-Network was trained over 15,000 episodes. The ϵ-greedy exploration rate started at 1.0 and decayed multiplicatively ($\times 0.9999$) each step, bottoming out at 0.05.

5.1 Summary Statistics

Table 1. DQN Training Final Metrics (Averaged over the Final 100 Episodes)

Metric	Observed Value
Total Training Episodes	15,000
Mean Episodic Reward	21.24
Goal Success Rate	47.47%
Wall Collision Rate	52.51%
Episode Timeout Rate	0.03%

5.2 Learning Curves

The progress of the reinforcement learning process is best visualized through the episodic reward curve and the survival steps curve shown in Figures 4 and 5.

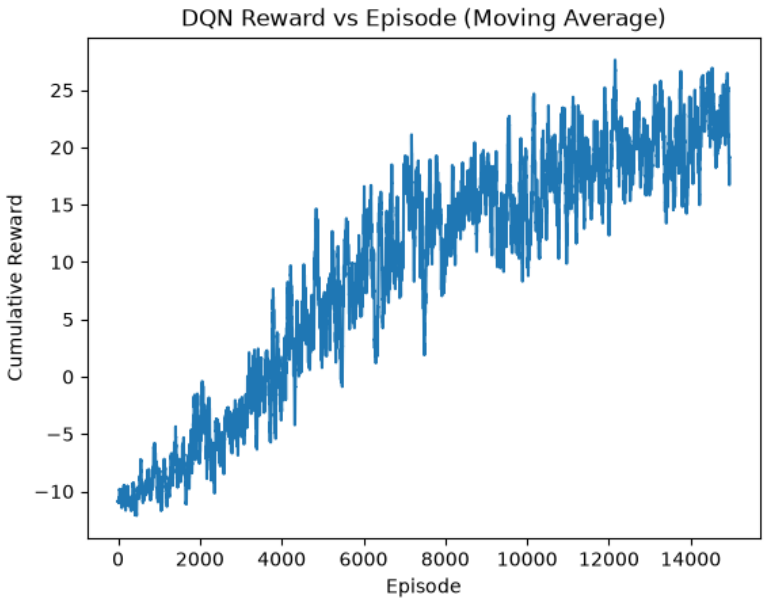

Figure 4. Cumulative reward vs. training episode. The upward trend reflects the agent's progressive discovery of higher-value state-action mappings, with visible plateaus corresponding to the failure modes analysed in Section 6.

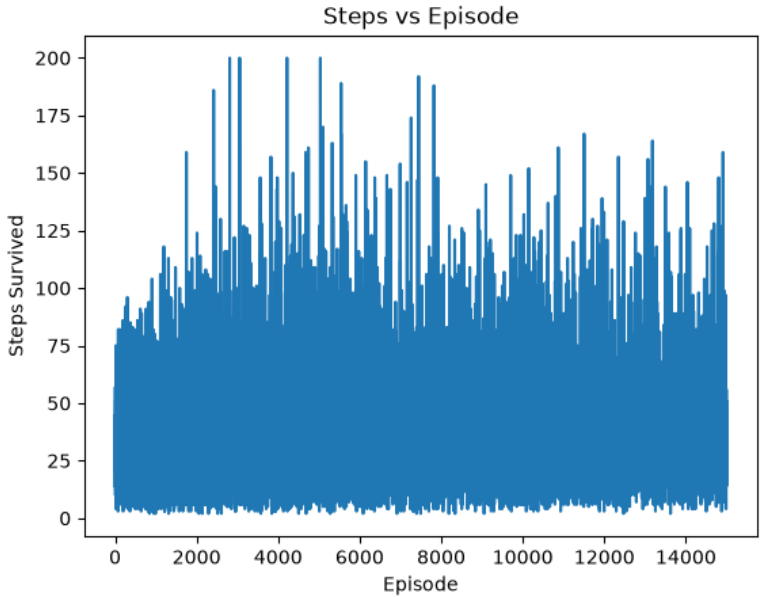

Figure 5. Steps survived vs. training episode. An initial increase indicates prolonged collision avoidance; the subsequent decrease reflects the agent converging on the shortest path to the goal.

6 Failure Analysis: Reward Plateaus and Vanishing TD Errors

Note: The mathematical analysis in this section reflects findings from initial failed training runs, which directly informed the robust architecture presented in Section 4.

6.1 Observed Failure Mode: Reward Plateau

A characteristic two-stage reward plateau pattern emerged during early DQN iterations:

- **Stage 1 — Early training:** Cumulative reward stabilized near -7 to -6 per episode. The agent learned to spin in circles, entirely avoiding wall collisions but making no progress toward the goal.
- **Stage 2 — Mid training:** Reward stabilized near $+51$ to $+52$ per episode. The agent navigated open space competently but consistently failed to locate or pass through the narrow wall corridor.

This behavior is not a software defect. It represents a well-characterized RL failure mode in which the agent converges to a locally optimal policy within a flat reward landscape that provides no gradient incentive for further improvement:

$$\mathbb{E}\left[\sum_{t=0}^{T} \gamma^t R_t\right] \approx \text{const.} \qquad \text{(no improvement across episodes)} \tag{18}$$

6.2 Mathematical Root Cause: Vanishing TD Error

DQN policy improvement is driven by the temporal-difference (TD) error:

$$\delta_t = r_t + \gamma \max_{a'} Q_{\phi^-}(s_{t+1}, a') - Q_\phi(s_t, a_t) \tag{19}$$

The standard Q-update takes the form:

$$Q(s,a) \leftarrow Q(s,a) + \alpha\, \delta_t \tag{20}$$

When the agent enters a stable loop policy (e.g., spinning infinitely), environment transitions become highly predictable and the TD error collapses to zero:

$$\delta_t \approx 0 \implies Q(s,a) \approx Q(s,a) \tag{21}$$

The network ceases to receive a useful learning signal, parameter updates become negligible, and the policy is effectively frozen.

6.3 Why ϵ-Greedy Exploration Fails to Escape the Plateau

At the terminal decay $\epsilon = 0.05$, the agent takes random actions 5% of the time. However, exploration rate is orthogonal to learning rate: if random actions do not lead to states with higher reward than the current policy's basin, the loss gradient remains near zero regardless:

$$\nabla_\phi L(\phi) \approx 0 \qquad \text{even under stochastic exploration} \tag{22}$$

The agent effectively samples noise within a reward plateau rather than ascending a reward gradient.

6.4 Primary Failure Causes and Mitigations

6.4.1 Cause 1: Absent or Insufficient Reward Shaping

Problem. A naive reward function produces large regions where $R(s) \approx \text{const}$, particularly when the agent reaches a stable sub-optimal trajectory. Without a smooth incentive gradient, the Bellman update cannot distinguish "better" states from "good enough" states:

$$\nabla_s R(s) \approx 0 \implies \text{no learning pressure toward } s^* \tag{23}$$

Mitigation. We augmented the reward function with potential-based reward shaping. By injecting the differential distance $d_t - d_{t+1}$ as a per-step reward, we supplied a continuous gradient toward the goal that pulled the agent out of the spinning local minimum.

6.4.2 Cause 2: Coarse or Non-Discriminative State Representation

Problem. An overly simplistic observation vector may be insufficient for the network to distinguish qualitatively different environmental configurations. Two states that appear identical in observation space but differ in long-horizon value produce conflicting Q-targets,

causing destructive gradient interference:

$$Q_\phi(s^{(1)}) \approx Q_\phi(s^{(2)}) \quad \text{even if} \quad V^*(s^{(1)}) \neq V^*(s^{(2)}) \tag{24}$$

Mitigation. We enriched the observation vector with the sine and cosine of the heading angle and the normalized relative bearing to the goal $\big(\Delta\theta \,/\, \pi\big)$. This augmented representation reduced value aliasing and improved credit assignment throughout training.

7 Future Improvements

To build upon the successes and address the remaining limitations of this architecture, we propose four concrete improvements:

1. **Prioritized Experience Replay (PER).** Instead of sampling uniformly from the replay buffer, PER samples transitions based on their absolute TD error $|\delta_t|$. This focuses computational resources on transitions where the model has the most to learn—critical collisions, surprising outcomes, and rare goal arrivals—significantly accelerating convergence. Implementation requires replacing uniform sampling with a sum-tree data structure that maintains and updates sample probabilities efficiently.

2. **Dueling DQN Architecture.** A Dueling DQN splits the network after the shared hidden layers into two streams: a state-value estimator $V(s)$ and an action-advantage estimator $A(s, a)$. This decoupling allows the agent to learn which states are inherently valuable independently of action impact, which is particularly effective in environments where many actions yield identical outcomes (e.g., moving forward in open space). The two streams are recombined at the output to reconstruct standard Q-values.

3. **Continuous Action Spaces via DDPG or PPO.** The current framework restricts the robot to three discrete, coarse movements. Upgrading to Deep Deterministic Policy Gradient (DDPG) or Proximal Policy Optimization (PPO) would enable smooth, continuous linear and angular velocities, allowing the robot to curve around obstacles and modulate speed dynamically based on proximity to walls.

4. **Adaptive Epsilon Decay.** The current schedule applies a fixed multiplicative decay ($\epsilon \times 0.9999$) regardless of agent performance. This risks prematurely suppressing exploration before the agent has discovered goal-reaching behavior. Linking the decay rate to a moving average of episode success rates would sustain exploration precisely when the policy is still sub-optimal and relax it only once consistent goal-reaching has been established.

8 Conclusion

This paper designed, implemented, and evaluated two distinct machine learning paradigms for autonomous edge robotics navigation. The Behavioral Cloning approach demonstrated that a compact, three-layer MLP is highly capable of mimicking hard-coded differential drive rules, making it an effective rapid-deployment method when a competent expert algorithm is available. The Deep Q-Network implementation revealed the broader potential of machine learning: by constructing a well-specified Markov Decision Process, implementing potential-based reward shaping, and employing Double-DQN target networks, the agent independently learned to interpret raw raycast data and navigate through a spatially constrained corridor. By rigorously analysing the mathematical root causes of early training failures—specifically vanishing temporal-difference errors within reward plateaus—we established a principled framework for reliable reinforcement learning training. The proposed future improvements, particularly continuous action spaces and prioritized experience replay, would bridge the gap between this simulated proof-of-concept and physical hardware deployment.

9 Data Availability

The full source code, iterative model checkpoints, exported network weights, and complete CSV datasets are publicly available at:

https://github.com/dev-prashanna/Spatial_Preception

This paper serves as an academic summary of the architecture and theoretical foundations; full experimentation details and raw data logs are accessible in the repository.

References

[1] Thrun, S., Burgard, W., & Fox, D. (2005). *Probabilistic Robotics*. MIT Press.

[2] Pomerleau, D. A. (1989). ALVINN: An autonomous land vehicle in a neural network. *Advances in Neural Information Processing Systems*, 1.

[3] Mnih, V., Kavukcuoglu, K., Silver, D., et al. (2015). Human-level control through deep reinforcement learning. *Nature*, 518(7540), 529–533.

[4] Sutton, R. S., & Barto, A. G. (2018). *Reinforcement Learning: An Introduction*. MIT Press.

[5] Ng, A. Y., Harada, D., & Russell, S. (1999). Policy invariance under reward transformations: Theory and application to reward shaping. *ICML*, 99, 278–287.

[6] Van Hasselt, H., Guez, A., & Silver, D. (2016). Deep reinforcement learning with double Q-learning. *Proceedings of the AAAI Conference on Artificial Intelligence*, 30(1).